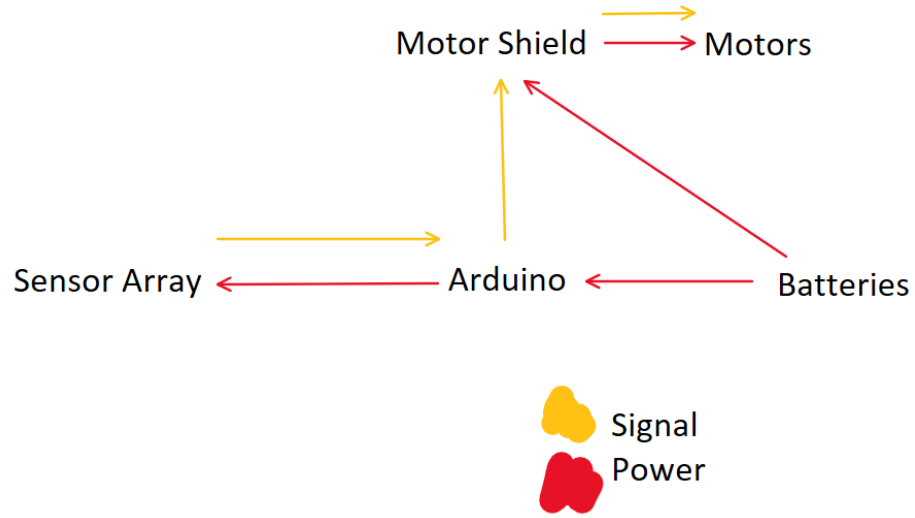
A Line Following Robot

Jonathan Sun and Ivry Frost

Introduction

In Mini Project 3, we made a line following robot that uses PID along with an array of IR sensors to control a car to follow a line. The Arduino R4 Wifi hosts a local web server that can be used to control the PID tuning constants, sensor floor/line threshold, and the base/maximum motor speed remotely while the car is moving.

Systems diagram

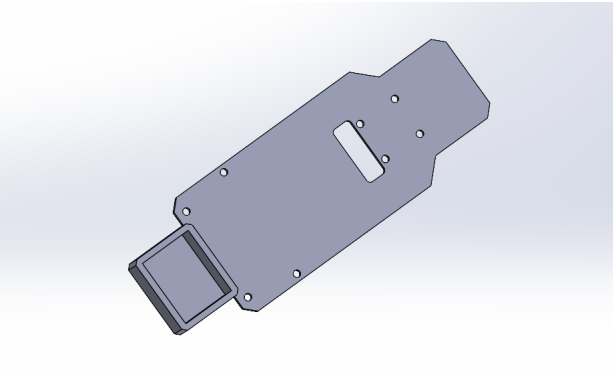


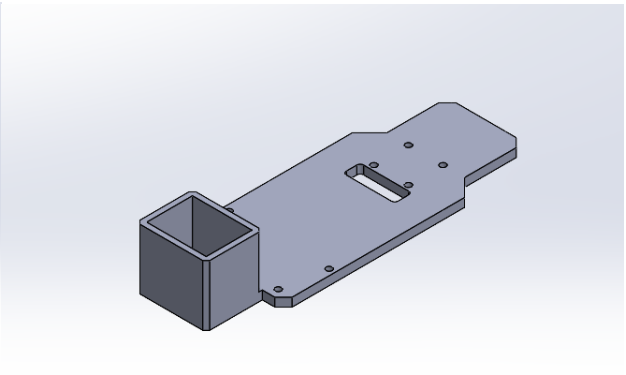
IR Sensor Calibration

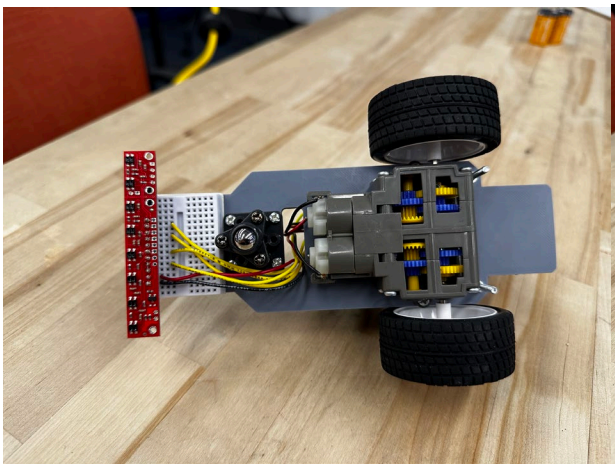
Our IR sensor array is calibrated using code that prints the IR sensor outputs to the serial monitor using values taken using the QTR IR Sensor Array Library. Using the library simplifies the process of taking data as we only needed to declare the sensor pins and the library outputs each sensor value scaled from 0-1000 when qtr.read is called. To determine if each sensor is over the ground or the line, we use a static threshold initialized within the code; however, for further tunability in variable lighting environments we included a tunable threshold parameter within our web interface so that the threshold can be edited wirelessly as the line follower is running.

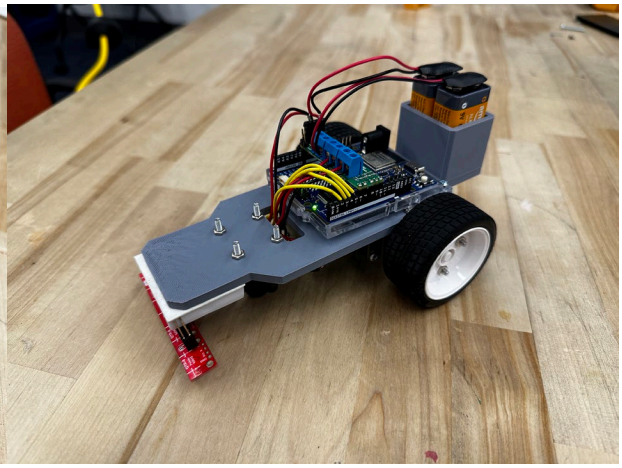
Mechanical Design and Documentation

Our mechanical design was relatively simple, with our focus being on making our setup small and efficient. Some notable components are a battery pack on the back of the chassis containing two 9V batteries, holes to mount our gearbox, holes to mount our Arduino, and mounting for our caster ball. We additionally included an open slot to allow our wiring to be easily passed through the center of the chassis for simplified wire management. The breadboard containing our sensor array is secured to the flat area in front of our chassis using the included adhesive on the breadboard. Below are two CAD screenshots of our chassis in SolidWorks, as well as photos of our final assembly including all hardware.





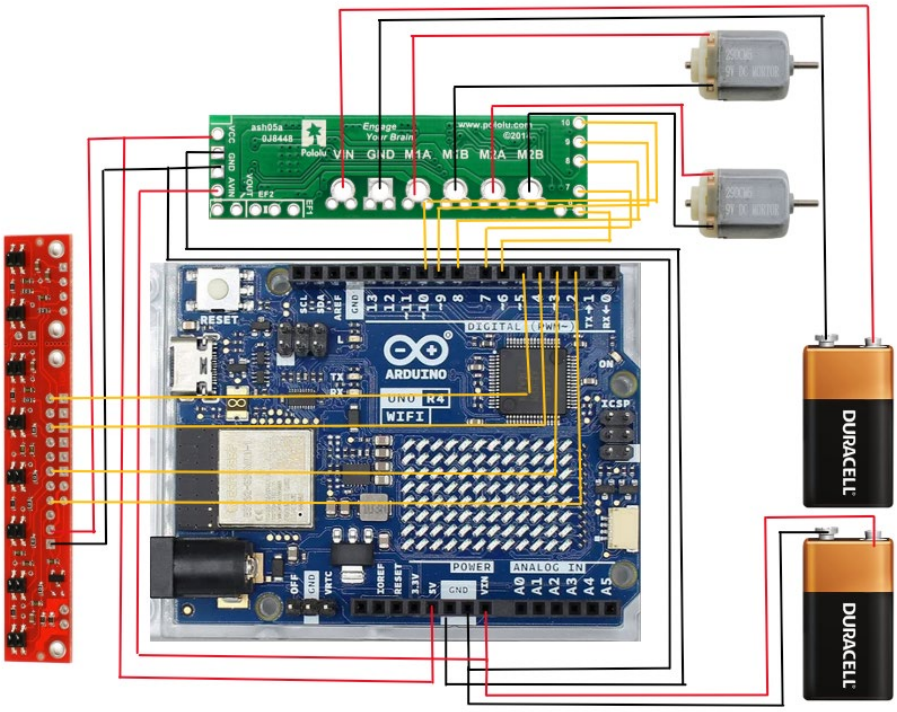




Our design is roughly 150mm front to back and makes use of most of the space. In addition, the front of our chassis is tilted slightly downwards to bring the sensors closer to the ground; this helps with sensor readings as the closer they are without touching the floor the larger the variance between the ground and line readings.

One thing that could be improved through further iteration is our center of gravity. As the batteries are mounted behind the main wheels, the center of gravity is only slightly ahead of the rear wheels. This causes it to occasionally pop a wheelie during hard acceleration or if the front wheel encounters a bump. In the future, shifting our battery mount forward could mitigate this.

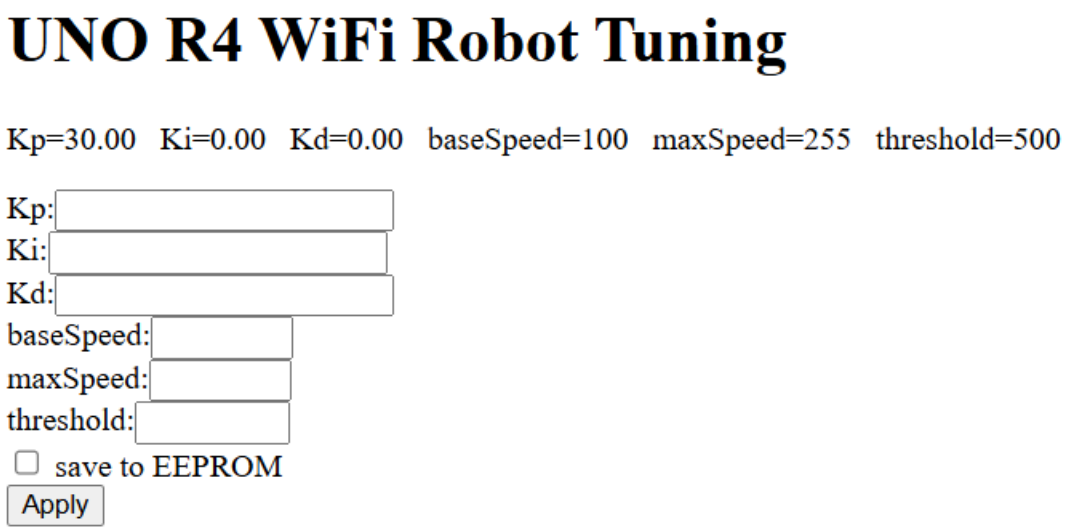
Circuit Diagram



Controller Logic

Our controller logic is centered around using sensor inputs and running them through PID to output values to each motor. Using the QTR-8C sensor array, we have 8 IR reflectance sensors on a single PCB. For the latest iteration of our code, we utilize the sensors in positions 1, 3, 6, and 8. This gives us two sensors on each side of the center line. The sensors are then assigned error values of –3, -1, 1, and 3 respectively, with sensors further from the sensor line having the larger/more negative value. The P (proportional) component is simply calculated by taking the sum of the error value of all sensors that detect the white line. If no sensors detect the line, the algorithm stores the last recorded sum error value and uses that instead, making the line follower continue turning in the direction it was

turning when it lost the line. For the I (integral) component, we calculate the total sum of all past error values and constrain it to positive and negative twenty thousand to avoid it from taking over the control algorithm. The D (derivative) component is calculated by subtracting the current error by the previous error value and then updating the previous error component. Each of these three components is multiplied by tuning constants Kp, Ki, and Kd and the sum of those products makes up the turn value. Using the turn value derived from PID, each motor receives a power value of base speed plus/minus (depending on left/right motor) and is constrained by top speed. To additionally make our line follower completely wireless, we host a web server on the Arduino R4 Wifi with a basic user interface that accepts values for Kp, Ki, Kd, base speed, max speed, and the line/ground threshold. In this video , you can see that when Jonathan pushes the enter button on his keyboard, he updates the speed values of our robot, making it to go faster.



To further ensure that we did not have to worry about any trailing wires from the line follower, we powered the motor shield with a 9V battery and the Arduino with another 9V battery. This ensured that we had sufficient power to both the board/sensors as well as the motors and would not suffer any brown outs from the motors drawing too much power.

Serial Interface

To get the data for our plot, we used a very simple serial monitor setup. As the loop was running, we had the serial monitor print in a csv readable format the values for the time (using millis), all the sensors, and the motors before going to the next line.

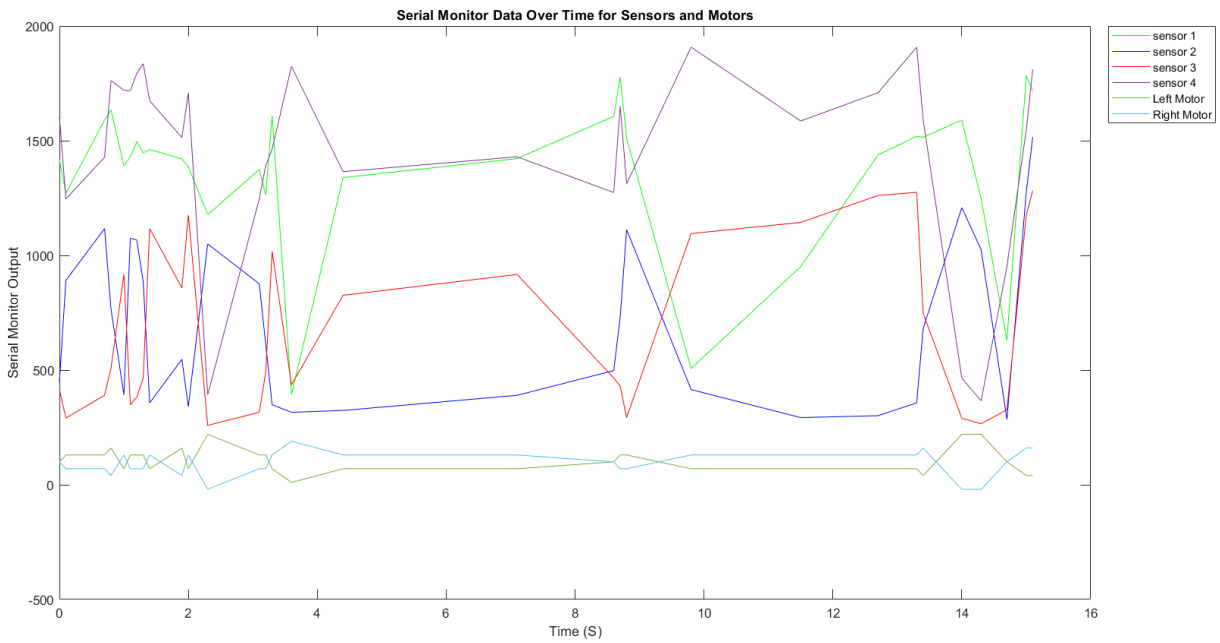
Example of one line of serial monitor output:

53100, 1418, 448, 415, 1604, 100, 100

These comma separated values can be very easily converted into a .csv file by copy pasting into notebook and saving as a csv. The first value represents the millis time, while the next four represent the IR sensor output, and the last two represent the left and then right motor values.

After importing our csv file into MATLAB, plotting our data was simple, as we just plotted our first column (time) against the other six columns. The serial print code snippet as well as the MATLAB code used to plot our data is in our code section, and the plot is below.

Plot With Motor Commands and Sensor Data



What we are looking for in our plot is similar trends for the IR sensors and for the motors to be inverse to each other. The motors hold true, with the left motor going up as the right motor goes down and vice versa. With the sensors, the pattern isn't incredibly strong, but the peaks and valleys do seem to be occurring around the same timestamps, which is to be expected.

Demo Videos:

Demo of our car completing the loop.

Demo of us updating the speed as while our robot is going around the loop.

Reflection

Something that is important to reflect on in this project is that we only used components we had independently sourced. Instead of a standard Arduino Uno, we used an R4 Wi-Fi, which has simple server hosting capabilities. This allowed us to send commands to our Arduino through a web server without having to stay physically connected to it, which improved our process greatly. In addition, we bought a gearbox, motors, motor shield, and sensors that were more effective than the provided parts. This allowed us to get by with much less adjustments to minimize noise than some other groups, as our equipment could perform more reliably. One could argue that this paints us as worse engineers over the course of this project, because we did not have to struggle as much as other groups. Another argument could be made that this project shows just how much good equipment matters. We don't have a firm conclusion to take away from the effects that improved parts caused, but it is something nice to think about.

Something that we did well in this project was thoroughly think out our process before proceeding. We thought out the mechanical design to an extent that, though we 3D printed our chassis with only 3 days to go, when we did print it, it was already perfect. The same goes for our code and circuitry processes. The story of this mini project is one of a lot of ideation, followed by a short burst of efficient work to finish. We are both happy with how this workflow went.

Despite finishing on time (with an extension), we both think we could have frontloaded a little bit more, to give ourselves more slack at the end and potentially remove the need for an extension.

All in all, we believe that the learnings we got from this project were more specific than the ones we got from mini project 2. For example, instead of learning how to use Millis as a

concept, we learned how to have an Arduino host a web server. In addition, we got to learn how to use a motor shield and have an Arduino control two wheels. The technical skills of this project stuck with the theme of specificity, and we both enjoyed narrowing down in this sense. In addition, we got to work on our workflow and teaming abilities, and we both believe that we are finding a good group projects groove, which will be helpful as we enter the PIE project.

Source Code:

```
#include <WiFiS3.h>
#include <EEPROM.h>
#include <QTRSensors.h>

// wifi credentials
const char* ssid = "OLIN-ROBOTICS";
const char* pass = "R0B0TS-RULE";

// init motor pins
const int A_PHASE = 7;   // left phase
const int A_PWM   = 9;   // left PWM
const int B_PHASE = 8;   // right phase
const int B_PWM   = 10;  // right PWM

// init pins for qtr sensor library
const uint8_t QTR_PINS[4] = {2, 3, 4, 5};
const uint8_t SensorCount = 4;

// init qtr sensors
QTRSensors qtr;
uint16_t qtrValues[SensorCount];

// floor/line threshold, lower values is line higher is floor
uint16_t sensorThreshold = 500;

// pid tuning values
struct Tunables {
```

```
  uint32_t magic;
  float Kp, Ki, Kd;
  int16_t baseSpeed;
  int16_t maxSpeed;
};
Tunables P = {0xA1B2C3D4, 0.18f, 0.0009f, 2.2f, 130, 255};

void saveTunables() { EEPROM.put(0, P); }
void loadTunables(){ Tunables t; EEPROM.get(0,t); if(t.magic==0xA1B2C3D4) P=t; }

// init integral + last error val for derivative
float   integral=0.0f;
int16_t lastError=0;

// init wifi server
WiFiServer server(80);

// setup web server
// parse float input function
float qf(const String& s,const char* key,float def){
  int i=s.indexOf(String(key)+"="); if(i<0) return def;
  int j=s.indexOf('&',i); String v=s.substring(i+strlen(key)+1,
j<0?s.length():j);
  return v.toFloat();
}
// parse integer input function
long  qi(const String& s,const char* key,long def){
  int i=s.indexOf(String(key)+"="); if(i<0) return def;
  int j=s.indexOf('&',i); String v=s.substring(i+strlen(key)+1,
j<0?s.length():j);
  return v.toInt();
}
// code for client side input/web client
void handleClient(WiFiClient& c){
  String req=c.readStringUntil('\r');
  while(c.available()){ if(c.read()== '\n') break; }

  if(req.startsWith("GET /set?")){
    P.Kp=qf(req,"kP",P.Kp); P.Ki=qf(req,"kI",P.Ki); P.Kd=qf(req,"kD",P.Kd);
    P.baseSpeed=(int16_t)qi(req,"base",P.baseSpeed);
    P.maxSpeed =(int16_t)qi(req,"max", P.maxSpeed);
    sensorThreshold = (uint16_t)qi(req,"th", sensorThreshold);
    if(req.indexOf("save=1")>0) saveTunables();
```

```
  }

  c.println("HTTP/1.1 200 OK\r\nContent-Type: text/html\r\nConnection:
close\r\n");
  c.println();
  c.println("<!doctype html><meta name='viewport' content='width=device-
width,initial-scale=1'>"
            "<title>Robot Tuning</title><h1>UNO R4 WiFi Robot Tuning</h1>");
  c.print("<p>Kp="); c.print(P.Kp);
  c.print("   Ki="); c.print(P.Ki);
  c.print("   Kd="); c.print(P.Kd);
  c.print("   baseSpeed="); c.print(P.baseSpeed);
  c.print("   maxSpeed="); c.print(P.maxSpeed);
  c.print("   threshold="); c.print(sensorThreshold);
  c.println("</p>");
  c.println("<form action='/set' method='get'>"
            "Kp:<input name='kP' step='0.001'><br>"
            "Ki:<input name='kI' step='0.0001'><br>"
            "Kd:<input name='kD' step='0.01'><br>"
            "baseSpeed:<input name='base' type='number' min='0' max='255'><br>"
            "maxSpeed:<input name='max' type='number' min='0' max='255'><br>"
            "threshold:<input name='th' type='number' min='0' max='1000'><br>"
            "<label><input type='checkbox' name='save' value='1'> save to
EEPROM</label><br>"
            "<button>Apply</button></form>");
  c.stop();
}

// motor control function
void setMotor(int16_t cmd, uint8_t phasePin, uint8_t pwmPin){
  digitalWrite(phasePin, cmd>=0 ? HIGH : LOW);
  analogWrite(pwmPin, abs(cmd));
}

// setup
void setup(){
  // init motor pinmodes
  pinMode(A_PHASE, OUTPUT); pinMode(A_PWM, OUTPUT);
  pinMode(B_PHASE, OUTPUT); pinMode(B_PWM, OUTPUT);

  // use qtr library to begin reading sensor array inputs
  qtr.setTypeRC();
  qtr.setSensorPins(QTR_PINS, SensorCount);
```

```
  qtr.setTimeout(2500);
  delay(250);

  loadTunables();

  Serial.begin(115200);
  delay(500);
  WiFi.begin(ssid, pass);
  unsigned long t0=millis();
  while(WiFi.status()!=WL_CONNECTED && millis()-t0<20000) delay(250);
  // print tuning web server ip
  if(WiFi.status()==WL_CONNECTED){ server.begin(); Serial.print("Web UI:
http://"); Serial.println(WiFi.localIP()); delay(3000);}
  else { Serial.println("Wi-Fi failed."); }
}

// loop
void loop(){
  if(WiFi.status()==WL_CONNECTED){
    WiFiClient client=server.available();
    if(client){
      unsigned long t0=millis();
      while(!client.available() && millis()-t0<2000) delay(1);
      if(client.available()) handleClient(client);
    }
  }

  // read qtr values and assign to array
  qtr.read(qtrValues);
  // set line/floor threshold value
  const uint16_t threshold = sensorThreshold;

  // check if each sensor detects the line
  bool line[SensorCount];
  for(uint8_t i=0;i<SensorCount;i++){
    line[i] = (qtrValues[i] <= threshold);
  }

  // assign error values to each sensor
  const int8_t W[SensorCount] = {-3, -1, 1, 3};
  int16_t error = 0;
  int16_t hits  = 0;
  // compute total error for all sensors
```

```
  for(uint8_t i=0;i<SensorCount;i++){
    if(line[i]){ error += W[i]; hits++; }
  }
  // if no sensor detects line use last error value
  if(hits == 0){
    error = (lastError > 0) ? 2 : (lastError < 0 ? -2 : 0);
  }

  // calculate pid values
  float Pterm = error;
  integral += error; integral = constrain(integral, -20000, 20000);
  float Iterm = integral;
  float Dterm = error - lastError; lastError = error;

  // use pid values and tuning constants from web server to compute turn value
  float turn = P.Kp*Pterm + P.Ki*Iterm + P.Kd*Dterm;

  // compute left and right motor speeds using turn value
  int16_t left  = constrain(P.baseSpeed + (int16_t)turn, -P.maxSpeed,
P.maxSpeed);
  int16_t right = constrain(P.baseSpeed - (int16_t)turn, -P.maxSpeed,
P.maxSpeed);

  // output time, each sensor value, and left/right motor output values every
100ms for graph
  if(millis()%100 == 0){
  Serial.print(millis()); Serial.print(", ");
  Serial.print(qtrValues[0]); Serial.print(", ");
  Serial.print(qtrValues[1]); Serial.print(", ");
  Serial.print(qtrValues[2]); Serial.print(", ");
  Serial.print(qtrValues[3]); Serial.print(", ");
  Serial.print(left); Serial.print(", ");
  Serial.println(right);
  }
  // set motor speeds
  setMotor(left,  A_PHASE, A_PWM);
  setMotor(right, B_PHASE, B_PWM);

}
```

MATLAB code to plot our data:

```
data = csvread("mp3_sensor_data.csv")
time = ((data(:, 1) / 1000) - 53.1);
sensor_1 = data(:, 2);
sensor_2 = data(:, 3);
sensor_3 = data(:, 4);
sensor_4 = data(:, 5);
motor_left = data(:, 6);
motor_right = data(:, 7);
figure()
plot(time, sensor_1, "g-", time, sensor_2, "b-", time, sensor_3, "r", time, sensor_4, time, motor_left, time, motor_right);
xlabel("Time (S)")
ylabel("Serial Monitor Output")
title("Serial Monitor Data Over Time for Sensors and Motors")
legend("sensor 1", "sensor 2", "sensor 3", "sensor 4", "Left Motor", "Right Motor", "Location","bestoutside")
```